

航空领域知识图谱构建与 RAG 应用 — 知识工程综合实践课说明文档

主题：基于大模型的航空/飞机领域知识图谱构建与 RAG (Retrieval-Augmented Generation) 问答系统实现

核心目标：把知识工程（知识获取、表示、存储、管理、问答、推理）贯通到一个可运行的工程中：学生分组采集权威数据、构建知识图谱（KG，三元组 ≥ 3000 条），并基于 KG + 文本构建 RAG 问答系统，最终提交可运行代码、实验报告与演示 PPT。

一、任务说明

每组 3-5 人按组提交。

- **数据/知识库：**KG 三元组（格式任选：CSV、JSON-LD），规模 ≥ 3000 三元组。
- **可运行代码：**至少包含用于导入 KG 的 Python 脚本（或直接提供 CSV 导入脚本），和用于启动 RAG 问答的演示脚本，**不要提交软件包**。
- **实验报告：**包含数据来源、爬取/清洗流程、知识建模、存储方案、检索与生成设计、结果与分析、存在问题与改进建议。
- **演示 PPT：**展示工作流程、关键实现、演示问答示例、结果分析与总结。

二、技术说明

大致流程可以参考链接：<https://zhuanlan.zhihu.com/p/9841668284>

1. 准备

实践开始之前需要先准备好编程环境，并学习简单的python 编程，推荐<https://www.runoob.com/python/python-tutorial.html>。

- 提前安装好 Python 环境（建议 Python 3.9+），pip、virtualenv 或 conda。
- 安装必备工具：`requests`，`beautifulsoup4`，`pandas`，`rdflib`，`py2neo`，`sentence-transformers`，`faiss-cpu` 或 `pymilvus`。

- 配置 Neo4j 与 Milvus（或本地 FAISS）用作存储与向量库。

2. 数据采集与初步清洗

1. 明确子主题与范围

- 航空是大域，建议聚焦两个或三个子域以保证深度与规模，例如：
 - 航空公司与飞机信息（机型、注册号、入役/退役、运营航线）
 - 机场与航线（机场代码、跑道、航站楼、城市）
 - 事故/事件/维护与法规摘要（事故摘要、原因分类、监管条目）
- 选择 1-2 个子域用于主要三元组生成，另加扩展数据以凑数。

去网上查找有没有同类型的知识图谱本体设计方法。

2. 权威数据源列举与优先级

- 官方机构：ICAO、IATA、FAA、各国民航局（CAAC 等）。
- 行业数据库：Airfleets、Planespotters、Aviation Safety Network（事故库）。
- 公开数据集：机场数据库（OpenFlights）、Wikipedia、航空公司官网、机型制造商官网（Boeing、Airbus）。

数据源只是建议，采集官方机构数据有难度，实在不行就维基，不建议使用大模型。

3. 数据采集实现

- 使用官方 API / CSV / 公开数据集或爬取网页，使用 `requests` + `BeautifulSoup`。

可以通过爬虫从官方网站获取数据，具体的爬虫编程教程可以在网上进行查找。推荐一个简单的爬虫笔记：

<https://www.cnblogs.com/JennyZhang-sharing/archive/2017/12/07/7998352.html>

- 建议保存原始文件（`raw/` 目录）与清洗脚本（`scripts/collect_*.py`）。

常见字段：机场（IATA、ICAO、名字、城市、经纬度、时区、跑道）、航线（航司、航班号、始发、到达、经停）、飞机（注册号、机型、制造商、序列号、座位数、引擎类型）、事故（时间、飞机、航班、地点、原因）。、

4. 数据清洗与解析 (pandas)

- 去重、统一编码 (UTF-8)、字段规范化 (日期格式 ISO)、实体命名规范 (例如: `Boeing 737-800` 统一写法)。
- 生成表格化中间产物: `entities.csv`, `relations.csv` (便于后续生成三元组)。

5. 实体识别与映射

- 可基于大模型抽取实体。

3. 知识建模 -> KG 构建与存储

1. 本体/模式设计 (Ontology / Schema)

- 先列出核心类 (Entity Types) 与关系 (Properties) 示例:
 - **类:** `Aircraft`, `Airport`, `Airline`, `AircraftType`, `Flight`, `Route`, `Incident`, `Manufacturer`, `Engine`, `Regulation`。
 - **关系:** `hasIATA`, `hasICAO`, `operatedBy`, `hasAircraftType`, `departsFrom`, `arrivesAt`, `usesRunway`, `involvesAircraft`, `occurredOnDate`, `hasCause`, `manufacturedBy`。
- 给出每个属性的数据类型与例子 (字符串、日期、数值、地理坐标)。



这里我提供的本体设计参考，特别是核心类示例不要出现歧义修改，例如我规定的 `AircraftType` 作为我们的飞机型号就不要使用 `AircraftModel` 等命名为您们的飞机型号的定义。

2. 从 CSV → 三元组 (示例)

- 若中间产物为 `entities.csv` (id, label, type, external_id), `relations.csv` (sub_id, pred, obj_id, provenance)

- 直接将其导出为三列 CSV: `subject,predicate,object`。

三元组样例:

- `"BKK","hasIATA","BKK"`
- `"Flight_AF123","operatedBy","Air France"`
- `"Boeing 737-800","manufacturedBy","Boeing"`

3. 存储选型

- **文件级**: CSV (三列) 或 TTL (rdflib) ——最轻量, 易提交。
- **图数据库**: Neo4j (使用 `py2neo` / `neo4j` 驱动)。
- **RDF 三元组存储**: 使用 `rdflib` 将三元组存为 TTL/Graph, 方便 SPARQL 查询演示。

4. 实体消歧与对齐

- 对同一实体用统一 ID (例如: `airport:ICAO_KJFK`), 并保存 `label`、`alias` 列。
- 若能提供外部链接 (Wikipedia URL、ICAO ID、IATA Code), 请保留在三元组中作为 `sameAs` 或外部属性。

4. RAG 实现、评估与演示

1. RAG 架构与组件

- **文档/知识切片**: 将 KG 的文本化描述 (节点属性、三元组文本) 与关联原始页面/报告拼接为检索文档。
- **向量化 / 检索层**: 使用 `sentence-transformers` 生成 embedding, 存入 FAISS (本地) 或 Milvus (服务)。
- **检索器 (Retriever)**: 向量检索 (近邻) + 基于 KG 的基准 SPARQL / 图查询作为辅助。
- **生成层 (Generator)**: 调用大模型 (可用开源 LLM 或云 API) 做基于检索上下文的回答生成。

2. 实现细节

- **文档切分**：按实体/关系或固定长度（例如 200-512 tokens）切分，保存 `docs.csv` (id,text,source)。
- **Embedding 示例 (Python)**：使用 `sentence-transformers`（例如 `all-MiniLM-L6-v2`）作为默认选择。
- **检索 + 生成简单流程**：
 1. 用户问题 → embedding
 2. 在向量库检索 Top-K 文档
 3. 将文档与问题拼接成 prompt，让 LLM 生成答案（并要求返回参考来源）。

3. 可替代与脱敏方案

- 使用本地开源 LLM（如 `llama.cpp` 前端）或使用规则化模板生成回答作为 demo。
- 或使用外部 API（如 OpenAI）Key

4. 评价与示例测试集

- 设计 10-20 道检验题（事实型、解释型、复杂推理型）。记录每题：问题、参考答案、系统返回、是否正确（人工标注），以及参考来源（检索到的文档 id）。
- 指标：准确率、检索覆盖率（参考答案中源文档是否被检索出）、回答可解释性（是否给出来源）。

5. 打包与演示准备

- README 中写明启动步骤与依赖，提供演示脚本。
- PPT 要包含：任务目标、数据来源样例、KG 结构图（节点-边示例）、RAG 流程图、两三个交互式问答演示、结果与问题、改进方向。

三、技术说明

A. 推荐文件夹结构（建议提交格式）

```
groupX_project/
├─ raw/           # 原始抓取数据
├─ data/         # 清洗后数据 (entities.csv, relations.csv, triples.csv, d
ocs.csv)
├─ kg/           # 最终 KG 文件 (triples.csv or graph.ttl)
├─ scripts/
│   ├─ collect_*.py
│   ├─ clean_*.py
│   ├─ build_triples.py
│   ├─ import_to_neo4j.py
│   └─ build_embeddings.py
├─ demo/
│   ├─ run_rag.py
│   └─ demo.sh
├─ 实验报告.pdf
├─ 展示PPT.pptx
└─ README.md
```

B. 常用 Python 包 (示例)

```
requests
beautifulsoup4
pandas
rdflib
py2neo
networkx
sentence-transformers
faiss-cpu
scikit-learn
spacy
jupyter
```

C. 小片段示例

- CSV → 三元组 (伪代码)

```
import pandas as pd
ents = pd.read_csv('data/entities.csv')
rels = pd.read_csv('data/relations.csv')
with open('kg/triples.csv','w',encoding='utf8') as f:
    for _,r in rels.iterrows():
        s = ents.loc[ents['id']==r['sub_id'],'label'].values[0]
        o = ents.loc[ents['id']==r['obj_id'],'label'].values[0]
        f.write(f'"{s}" {r["pred"]} "{o}"\n')
```

- **rdflib 写入 TTL (示例)**

```
from rdflib import Graph, URIRef, Literal, Namespace
g = Graph()
EX = Namespace('http://example.org/avi/')
# 示例插入
g.add((EX['Boeing737-800'], EX['manufacturedBy'], Literal('Boeing')))
# 保存
g.serialize(destination='kg/graph.ttl', format='turtle')
```

- **sentence-transformers Embedding (示例)**

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')
emb = model.encode(['文本1','文本2'])
# 保存 emb 与 doc id, 后续建立 FAISS 索引
```

四、帮助与常见问题 (FAQ)

- **三元组不够怎么提速?** : 扩大数据源 (例如把每个航班历史拆为多条事件记录)、引入更多属性 (发动机型号、生产年份、服役日期、航线里程数等), 或把属性拆为独立三元组。
- **可以只交 CSV 吗?** : 可以, 但请附上导入脚本与说明。